JeremyDEZETREE_TP3

October 18, 2025

1 TP 3 "Régression polynomiale / Régularisation"

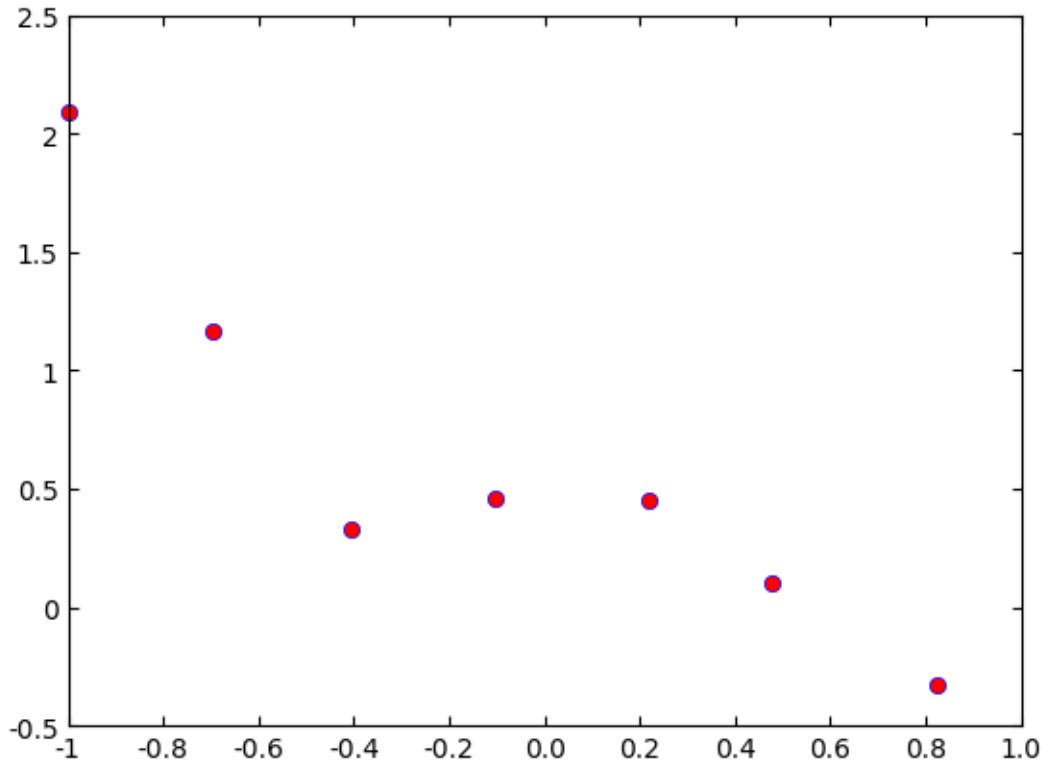
M2 IWOCS, Apprentissage automatique

1.1 Données du problème

Télécharger l'archive correspondant aux données de l'exercice 3/TP 3, décompactez-la. cette archive contient deux fichiers, un pour les données d'entrée x qui sont ici à une dimension, et l'autre pour les données de sorties y à une dimension également. Tracer un graphique représentant le nuage de points défini par ces données dans le style du celui qui est représenté dans la figure suivante.

[14]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
%matplotlib inline

x = np.loadtxt('ex3Linx.dat')
y = np.loadtxt('ex3Liny.dat')

graphe = sns.scatterplot(x=x, y=y, color="red", edgecolor="blue", clip_on=False)
def styliserGraphe(graphe, xlim=[-1, 1], ylim=[-0.5, 2.5], multiple=0.2):
    graphe.set_xlim(left=xlim[0], right=xlim[1])
    graphe.set_ylim(bottom=ylim[0], top=ylim[1])

    def formatIntergers(value, tick_number):
        return f'{int(value)}' if value % 1 == 0 else f'{value:.1f}'
    graphe.xaxis.set_major_locator(ticker.MultipleLocator(multiple))
    graphe.xaxis.set_major_formatter(ticker.FuncFormatter(formatIntergers)) #␣
 ↪Il me reste des décimales, je ne sais pas pourquoi.
    graphe.yaxis.set_major_formatter(ticker.FuncFormatter(formatIntergers))

    graphe.spines['right'].set_visible(True)
    graphe.spines['top'].set_visible(True)
    graphe.tick_params(axis='x', top=True, direction='in',labeltop=False)
    graphe.tick_params(axis='y', right=True, direction='in', labelright=False)
    return graphe
```

```
graphe = styliserGraphe(graphe)
```



1.2 Analyse du problème

On propose de modéliser ces données par une fonction hypothèse polynomiale de degré 5 :

$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + +\theta_3 x^3 + \theta_4 x^4 + \theta_5 x^5$$

Comme expliqué dans le cours, cela revient à considérer 6 descripteurs constituant les différentes puissances de x dans le polynôme. Le problème reste cependant linéaire par rapport à ces descripteurs.

Le degré du polynôme étant 5 et le nombre de points initiaux 7, le risque de sur-apprentissage (over-fitting) est grand. Il est donc nécessaire de *régulariser* le modèle.

Pour cela on va corriger le calcul de la fonction de coût en utilisant une variante de la régularisation de Ridge vu en cours et qui s'écrit :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x_i) - y_i\right)^2 + \lambda \sum_{j=1}^{n} \theta_j^2$$

Le paramètre de régularisation λ va permettre d'éviter des valeurs importantes de θ qui feraient augmenter la valeur de la fonction de coût. A noter que la composantes θ_O n'intervient pas dans la sommation devant λ.

[15]:
```
def hypothese(teta, X):
    return X@teta
```

1.3 Modification des formulations matricielles pour intégrer le terme de régularisation de type Ridge

L'introduction de la régularisation intervient dans la fonction de calcul du coût et dans le calcul de la minimisation de la fonction coût avec la méthode de gradient

1.3.1 Calcul du coût

Formule du coût avec la régularisation proposée dans l'exercice (variante de la régularisation de Ridge) :

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta(x_i) - y_i\right)^2 + \lambda\sum_{j=1}^{n}\theta_j^2$$

Comme vu précédemment, la première partie de cette formule (avant le terme de régularisation)

$$\frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta(x_i) - y_i\right)^2$$

s'écrit en notation matricielle

$$\frac{1}{2m}\left[h_\theta(X) - Y\right]^t\left[h_\theta(X) - Y\right]$$

avec

$$\begin{aligned}
\text{avec} \quad h_\theta(X) &= X\theta && (1)\\
&= \begin{bmatrix} 1 & x_{11} & \cdots & x_{1n} \\ \vdots & \vdots & & \vdots \\ 1 & x_{m1} & \cdots & x_{mn} \end{bmatrix}\begin{bmatrix}\theta_0 \\ \vdots \\ \theta_n\end{bmatrix} && (2)\\
\text{et} \quad Y &= \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} && (3)
\end{aligned}$$

On doit alors ajouter à la fonction coût, le terme $\lambda\sum_{j=1}^{n}\theta_j^2$ (attention : on a exclu θ_0). En notation matricielle, cette expression s'écrit

$$\lambda[\theta_]^t[\theta_] \quad \text{où} \quad \theta_ = \begin{bmatrix} 0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

Donc au final $J(\theta)$ s'écrit en notation matricielle

$$J(\theta) = \frac{1}{2m}\left[X\theta - Y\right]^t\left[X\theta - Y\right] + \lambda[\theta_]^t[\theta_]$$

1.3.2 Calcul de la minimisation de la fonction de coût régularisée avec la méthode de gradiant

A chaque itération, on calcule une nouvelle valeur de θ^* à partir de θ telle que $J(\theta^*) < J(\theta)$.

La méthode de gradient correspond au calcul suivant :

$$\theta^* = \theta - \alpha \nabla J(\theta)$$

$$\begin{cases} \theta_0^* & = \theta_0 - \alpha \dfrac{\partial}{\partial \theta_0} J(\theta) \\ \vdots & \\ \theta_k^* & = \theta_k - \alpha \dfrac{\partial}{\partial \theta_k} J(\theta) \\ \vdots & \\ \theta_k^* & = \theta_k - \alpha \dfrac{\partial}{\partial \theta_k} J(\theta) \end{cases}$$

$$\frac{\partial}{\partial \theta_k} J(\theta) = \frac{\partial}{\partial \theta_k} \left(\frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x_i) - y_i\right)^2 + \lambda \sum_{j=1}^{n} \theta_j^2 \right) \tag{4}$$

$$= \frac{1}{m} \left(\sum_{i=1}^{m} \left(h_\theta(x_i) - y_i\right) x_{ik} \right) + 2\lambda\theta_k \tag{5}$$

Le dernier terme du membre droit de l'équation $2\lambda\theta_k$ vaut 0 si $k = 0$ en raison de la formule de régularisation utilisée.

La formulation matricielle s'écrit ainsi (en utilisant que $h_\theta(X) = X\theta$) :

$$\theta^* = \theta - \alpha \left[\frac{1}{m} X^t [X\theta - Y] + 2\lambda\theta_- \right]$$

[16]:
```
def coutMatriciel(teta, X, Y, lambdaFactor) -> float:
    """Calcule J(teta) avec régularisation Ridge"""
    m = len(X)
    predictions = hypothese(teta, X)
    erreur = predictions - Y
    coutDeBase = (1 / (2 * m)) * (erreur.T @ erreur)

    tetaSans0 = teta.copy()
    tetaSans0[0] = 0
    regularisation = lambdaFactor * (tetaSans0.T @ tetaSans0)
    return coutDeBase[0, 0] + regularisation[0, 0]

def descenteGradient(X, Y, lambdaFactor, alpha=0.1, iterations=100000,␣
 ↪tolerance=1e-12, retirerTeta_=False):
    """Trouve les teta optimaux par descente de gradient"""
    m = len(X)
```

```
    n = X.shape[1]
    teta = np.zeros((n, 1))
    for i in range(iterations):
        predictions = hypothese(teta, X)
        erreur = predictions - Y
        gradient = (1 / m) * (X.T @ erreur)
        if not retirerTeta_:
            tetaSans0 = teta.copy()
            tetaSans0[0] = 0
            gradient += (2 * lambdaFactor) * tetaSans0
        else:
            gradient += (2 * lambdaFactor) * teta
        tetaStar = teta - alpha * gradient
        if abs((coutMatriciel(tetaStar, X, Y, lambdaFactor) -␣
↪coutMatriciel(teta, X, Y, lambdaFactor))/coutMatriciel(teta, X, Y,␣
↪lambdaFactor)) < tolerance:
            break
        teta = tetaStar

    return teta
```

1.4 Discussion sur le choix de la régularisation de type Ridge utilisée

Afin de comprendre l'intérêt du choix de la régularisation utilisée ici, à savoir une régularisation Ridge sans prendre le premier terme θ_0, on pourra comparer les résultats que l'on obtiendra dans la mise en ouvre proposée ci-dessous avec une régularisation de Ridge complète et intégrant θ_0.

1.5 Mise en oeuvre

Il s'agit d'adapter l'algorithme de régression linéaire écrite sous forme matricielle et modifié à partir des formules précédents pour intégrer le terme de régularisation.

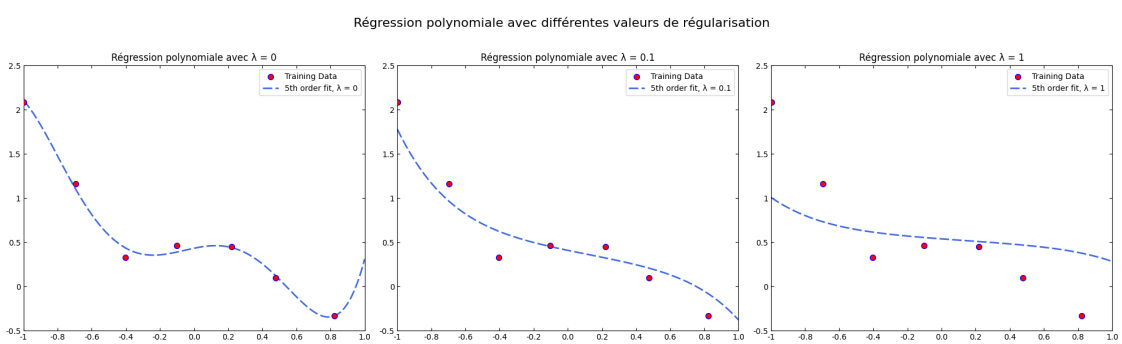
Comme cela est expliqué dans le cours, on va ramener le problème de regression polynomiale univarié à un problème de régression linéaire multivarié, en constituant une matrice X de la façon suivante à partir du vecteur colonne x de la donnée initiale :

`X=np.block([[xone,x,x**2,x**3,x**4,x**5]])`

Réaliser cette régression pour $\lambda = 0, \lambda = 1, \lambda = 10$, puis tracer la courbe résultante sur le même graphique que les données. On doit alors obtenir une figure comparable à la suivante :

[21]:
```
lambdas = [0, 0.1, 1]
m = len(x)
X = np.column_stack([np.ones(m), x, x**2, x**3, x**4, x**5])
Y = y.reshape(-1, 1)
xLine = np.linspace(-1, 1, 100)
XLine = np.column_stack([np.ones(100), xLine, xLine**2, xLine**3, xLine**4,␣
↪xLine**5])
```

```
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(20, 6))
fig.suptitle('Régression polynomiale avec différentes valeurs de␣
↪régularisation', fontsize=16)

for idx, lam in enumerate(lambdas):
    graphe = axes[idx]

    graphe.scatter(x, y, color='red', edgecolor='blue', s=50, label='Training␣
↪Data', zorder=5)
    tetaOptimal = descenteGradient(X, Y, lam, alpha=0.05, iterations=100000)
    yPrediction = hypothese(tetaOptimal, XLine)

    graphe.plot(xLine, yPrediction, dashes=[6,2], color="royalblue",␣
↪linewidth=2,
                label=f"5th order fit,   = {lam}")

    graphe = styliserGraphe(graphe)
    graphe.set_title(f'Régression polynomiale avec   = {lam}')
    graphe.legend()

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig('comparaison_regression_lineaires_reguler.png')
plt.show()
```



1.6 Compromis biais-variance

A partir de l'article wikipedia sur le compromis biais-variance en apprentissage : https://fr.wikipedia.org/wiki/Dilemme_biais-variance qui est référencé dans le cours, déduire une valeur de λ qui réalise le bon compromis entre biais du modèle et variance du modèle. On pourra faire les mêmes calculs que précédemment avec chaque valeur λ du vecteur de valeurs $\lambda = [0, 0.5, 1, ..., 5]$ et obtenir une série de courbes comme décrites ci-dessous :

Tracer les courbes de calcul du biais et de la variance du modèle pour chacune de ces valeurs et analyser le résultat pour conclure.

Rappel de la formule de décomposition de l'erreur décrite dans le cours :

- le biais se calcule par

$$B(h(x)) = E[(h - f)(x)]$$

- la variance se calcule par

$$Var(h(x)) = E[h^2(x)] - (E[h(x)])^2$$

On définit ensuite l'*erreur de prédiction attendue* par

$$Err(h(x)) = B^2(h(x)) + Var(h(x)) + \sigma^2$$

où E correspondra ici à l'espérance mathématique sur l'ensemble des exemples d'apprentissage utilisé.

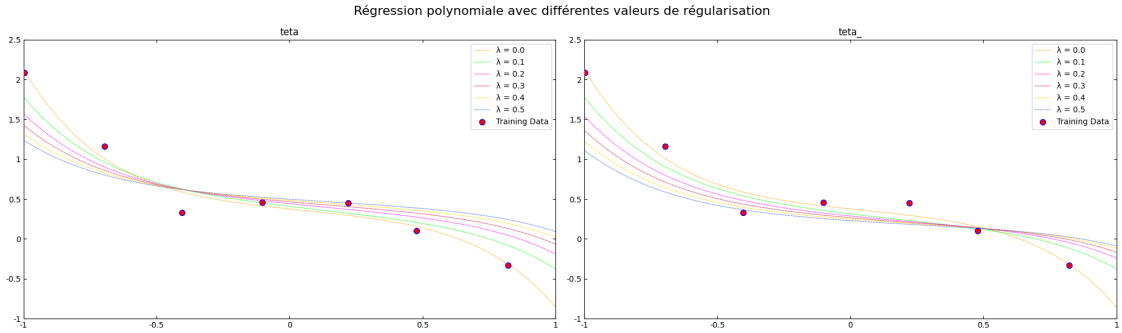
L'intersection des courbes pour ces deux grandeurs en fonction de λ ainsi que l'erreur globale indiquera la valeur du compromis biais-variance.

Remarque : Comme précisé dans le cours, l'analyse biais-variance se fait généralement sur plusieurs échantillonages ou jeux de données x (on en a qu'un seul ici car il y a trop peu de points expérimentaux) à partir desquels on obtient autant de fonctions hypothèses sur lesquels on doit moyenner les résultats. Le cas présenté est donc trop réduit pour cette analyse. Le résultat obtenu est de ce fait peu précis, voire peu fiable ici.

[24]:
```
lambdas = [i*0.1 for i in range(6)]
plt.clf()
fig, graphes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6))
fig.suptitle('Régression polynomiale avec différentes valeurs de␣
↪régularisation', fontsize=16)
colors = ["orange", "lime", "magenta", "crimson", "gold", "royalblue", "aqua",␣
↪"darkred", "lightseagreen", "mediumslateblue", "chocolate"]
avecTeta_ = [False, True]
for ids, boolean in enumerate(avecTeta_):
    graphe = graphes[ids]
    for idx, lam in enumerate(lambdas):
        tetaOptimal = descenteGradient(X, Y, lam, alpha=0.0005,␣
↪iterations=100000, retirerTeta_=boolean)
        yPrediction = hypothese(tetaOptimal, XLine)

        graphe.plot(xLine, yPrediction, dashes=[1,1], color=colors[idx],␣
↪linewidth=1,
                    label=f"  = {lam:.1f}")

    graphe.set_title(f'teta_') if boolean else graphe.set_title(f'teta')
    graphe.scatter(x, y, color='red', edgecolor='blue', s=50, label='Training␣
↪Data', zorder=5)
    graphe = styliserGraphe(graphe, ylim=[-1, 2.5], multiple=0.5)
```

```
    graphe.legend()

plt.tight_layout(rect=[0, 0, 1, 1])
```

```
<Figure size 640x480 with 0 Axes>
```



1.7 Éléments de calcul du compromis biais-variance proposé

Le Biais défini par la formule

$$B(h(x)) = E[(h - f)(x)]$$

se calcule en faisant la moyenne (l'espérance se simplifie en moyenne, comme expliqué dans le cours) des coefficients du vecteur $y - h(x)$. Ce calcul s'obtient en appelant la fonction `mean`de `numpy`, à laquelle on passera en paramètre le vecteur/matrice $y - h(x)$.

La variance se calcule par la formule

$$Var[h(x)] = E[(h(x))^2] - (E[h(x)])^2$$

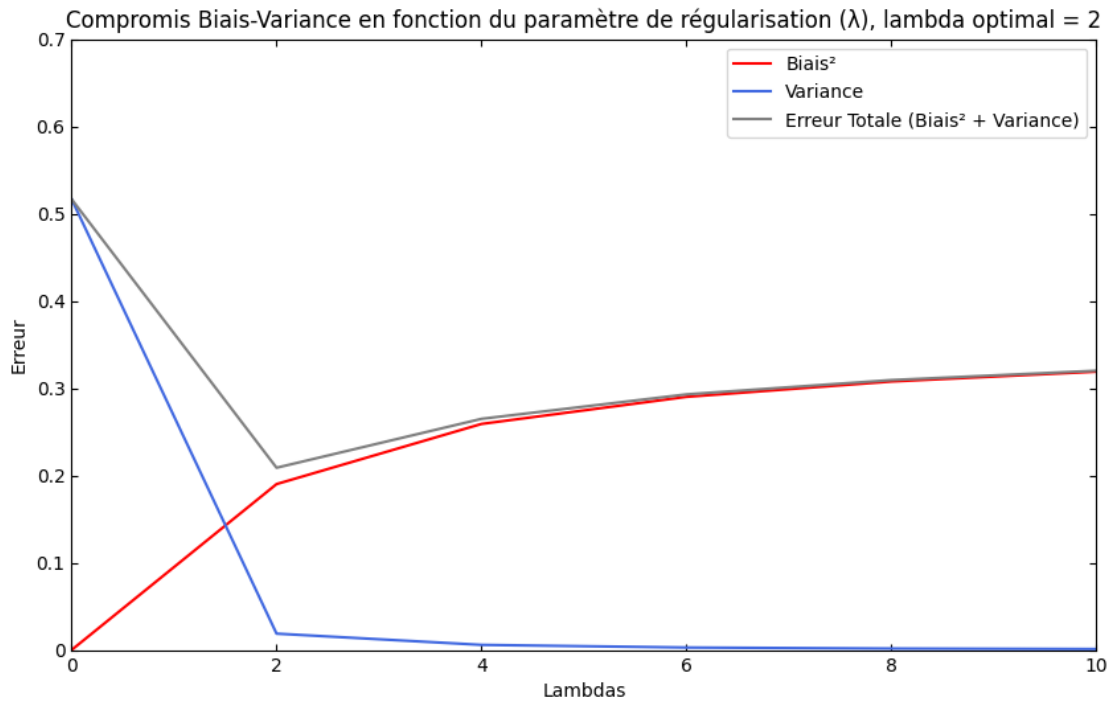
Là encore on simplifie l'espérance mathématique E par un simple calcul de moyenne.

L'expression $Var[h(x)]$ se calculera simplement en appelant la fonction `var`de `numpy`, à laquelle on passera en paramètre le vecteur/matrice $h(x)$.

Comme cela est mentionné, ce calcul de variance est une "interprétation" simplifiée de la définition du cours qui nécessite de mesurer la dispersion des valeurs de $h(x)$ par rapport à un ensemble d'échantillonages que l'on ne fait pas ici dans le TP. Calculer la "vraie" variance nécessiterait de mettre en place cet échantillonage et compliquerait singulièrement le TP. L'"interprétation" fait pour cet exercice est simple à calculer et produit des résultats exploitables et semblables au comportement de la "vraie" variance pour les données fournies.

Les courbes du biais et de la variance sont tracées dans cet exercice en prenant en abscisses les valeurs de λ. Mais comme la "complexité" du modèle est plus grande lorsque λ est petit, les sens de variations des courbes seront à l'inverse de la figure donnée en cours. Ici la courbe du biais va croitre alors que celle de la variance va décroitre.

[6]:
```
# Définition de la plage de valeurs pour lambda
lambdas = np.array([i*2 for i in range(6)])

# Préparation de la matrice de features
m = len(x)
X = np.column_stack([np.ones(m), x, x**2, x**3, x**4, x**5])
Y = y.reshape(-1, 1)
xLine = np.linspace(-1, 1, 100)
XLine = np.column_stack([np.ones(100), xLine, xLine**2, xLine**3, xLine**4,␣
 ↪xLine**5])

# Listes pour stocker les métriques
bias_carre_values = []
variance_values = []
total_error_values = []

for lam in lambdas:
    tetaOptimal = descenteGradient(X, Y, lam, alpha=0.0005, iterations=50000,␣
 ↪tolerance=1e-9, retirerTeta_=True)
    predictions_train = hypothese(tetaOptimal, X)

    biais = np.mean(Y - predictions_train)
    biais_carre = biais**2
    variance = np.var(predictions_train)
    total_error = biais_carre + variance

    bias_carre_values.append(biais_carre)
    variance_values.append(variance)
    total_error_values.append(total_error)

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(lambdas, bias_carre_values, label='Biais²', color='red')
ax.plot(lambdas, variance_values, label='Variance', color='royalblue')
ax.plot(lambdas, total_error_values, label='Erreur Totale (Biais² + Variance)',␣
 ↪color='gray')
ax = styliserGraphe(ax, ylim=[0, 0.7], xlim=[0, 10], multiple=2)
ax.set_title('Compromis Biais-Variance en fonction du paramètre de␣
 ↪régularisation ( ), lambda optimal = 2')
ax.set_xlabel('Lambdas')
ax.set_ylabel('Erreur')
ax.legend()
plt.show()
```

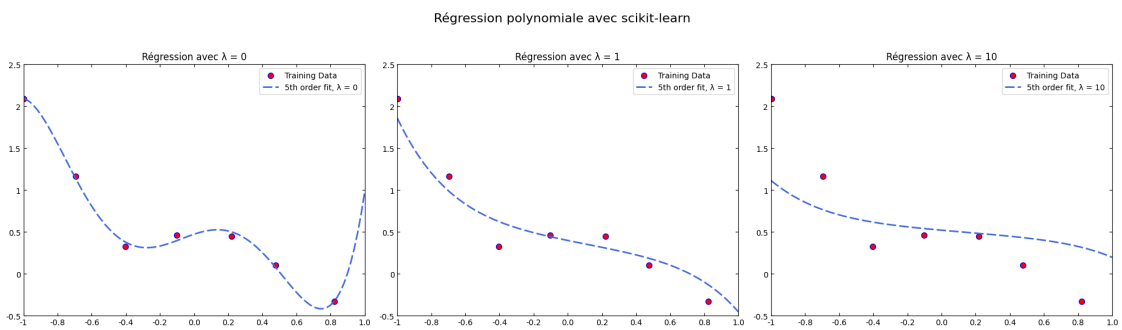


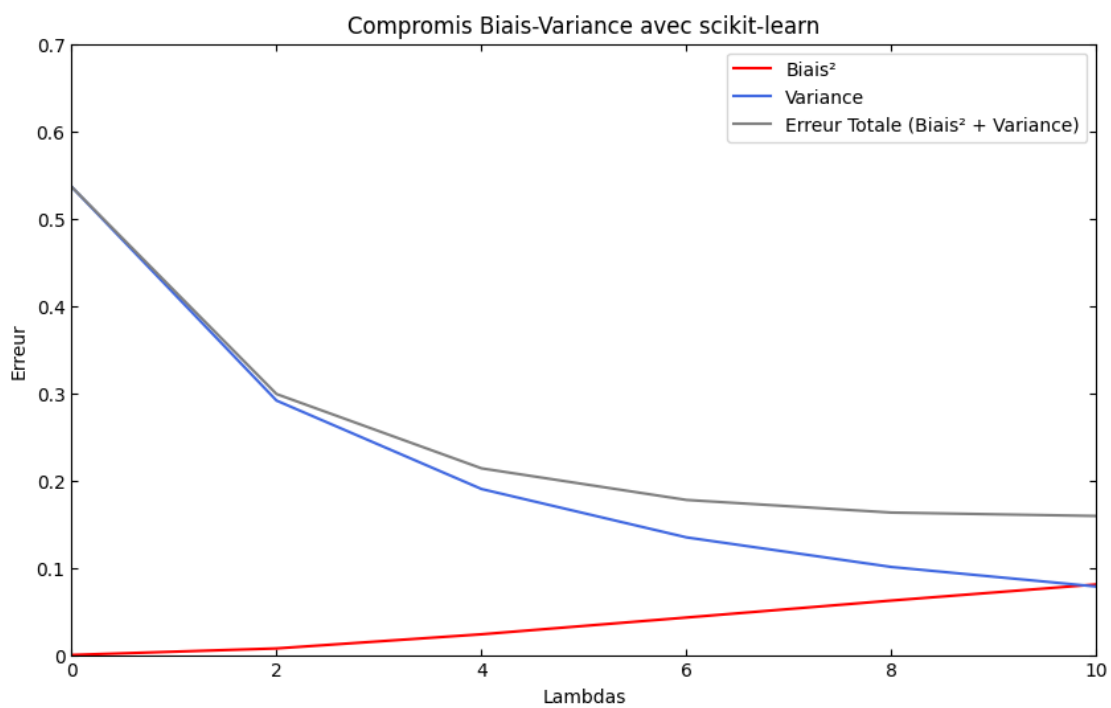
1.8 Reprise du développement en utilisant la bibliothèque `scikit-learn`

Reprendre le programme en Python en remplaçant les fonctions de calcul de régression par les fonctions prédéfinies de la bibliothèse `scikit-learn` qui implémente les techniques de régularisation connues dont celle de Ridge sur laquelle s'appuie la méthode proposée dans le TP.

[9]:
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline

lambdas = [0, 1, 10]
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(20, 6))
fig.suptitle('Régression polynomiale avec scikit-learn', fontsize=16)

X_reshaped = x.reshape(-1, 1)
x_plot = np.linspace(-1, 1, 100).reshape(-1, 1)

for idx, lam in enumerate(lambdas):
    ax = axes[idx]

    # Création du pipeline: Créer les features polynomiales et appliquer la␣
↪régression Ridge
    # Le paramètre alpha de Ridge correspond à lambda
```

```
    model = make_pipeline(PolynomialFeatures(degree=5), Ridge(alpha=lam,␣
 ↪solver='cholesky'))
    model.fit(X_reshaped, y)
    y_plot = model.predict(x_plot)

    ax.scatter(x, y, color='red', edgecolor='blue', s=50, label='Training Data')

    ax.plot(x_plot, y_plot, dashes=[6, 2], color="royalblue", linewidth=2,
            label=f"5th order fit,   = {lam}")

    ax = styliserGraphe(ax)
    ax.set_title(f'Régression avec   = {lam}')
    ax.legend()

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



[8]:
```
# --- Calcul du biais et de la variance ---
lambdas = np.array([i*2 for i in range(6)])
X_reshaped = x.reshape(-1, 1)

bias_carre_values = []
variance_values = []
total_error_values = []

for lam in lambdas:
    # Création et entraînement du modèle
    model = make_pipeline(PolynomialFeatures(degree=5), Ridge(alpha=lam,␣
 ↪solver='cholesky', fit_intercept=False))
    model.fit(X_reshaped, y)

    # Prédictions sur les données d'entraînement pour l'analyse
    predictions_train = model.predict(X_reshaped)
```

```
    # Calcul du biais et de la variance comme dans votre notebook
    biais = np.mean(y - predictions_train)
    biais_carre = biais**2
    variance = np.var(predictions_train)
    total_error = biais_carre + variance

    bias_carre_values.append(biais_carre)
    variance_values.append(variance)
    total_error_values.append(total_error)

# --- Affichage des courbes de compromis ---
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(lambdas, bias_carre_values, label='Biais²', color='red')
ax.plot(lambdas, variance_values, label='Variance', color='royalblue')
ax.plot(lambdas, total_error_values, label='Erreur Totale (Biais² + Variance)',␣
 ↪color='gray')
ax = styliserGraphe(ax, ylim=[0, 0.7], xlim=[0, 10], multiple=2)
ax.set_title('Compromis Biais-Variance avec scikit-learn')
ax.set_xlabel('Lambdas')
ax.set_ylabel('Erreur')
ax.legend()

plt.show()
```